

TM Simulation - trajectoire de fusée:

Tests et étape 2

Rendu du 19.04.26

1 Introduction et théorie de l'étape 1

1.1 Choix du phénomène

Tout d'abord, j'ai choisi ce phénomène parce qu'il est intéressant de pouvoir visualiser par le moyen d'une simulation les différentes forces qui agissent sur une fusée. Une simulation permet aussi de prédire le comportement au fil du temps selon les propriétés du véhicule (dont la masse, par exemple). Enfin, j'ai principalement choisi de faire cette simulation par intérêt pour l'aérodynamique.

1.2 Décomposition en plusieurs étapes

1. Trajectoire simple avec les 4 forces aérodynamiques
2. Prise en compte des étages, de la gravitation
3. Prise en compte de la pression atmosphérique et autres détails (gravity turn)

1.3 Théorie

La trajectoire dépend de 4 forces:

La poussée:

$$T = \dot{m}\Delta v$$

La gravité:

$$W = mg$$

La portance:

$$L = \frac{1}{2}\rho v^2 AC_l$$

La traînée:

$$D = \frac{1}{2} \rho v^2 A C_d$$

Légende:

$\dot{m}$ = débit de masse

ρ = masse volumique

C_l = coefficient de portée

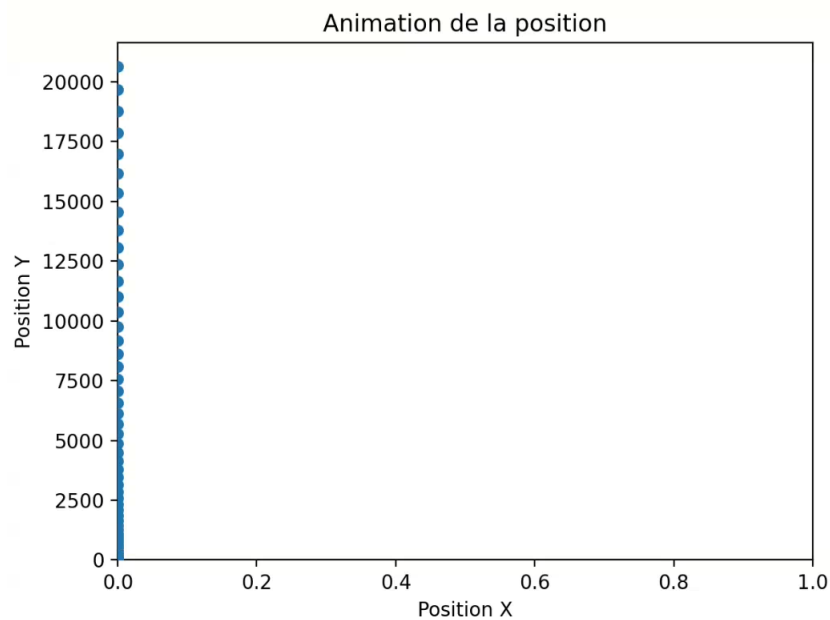
C_d = coefficient de traînée

v = vitesse

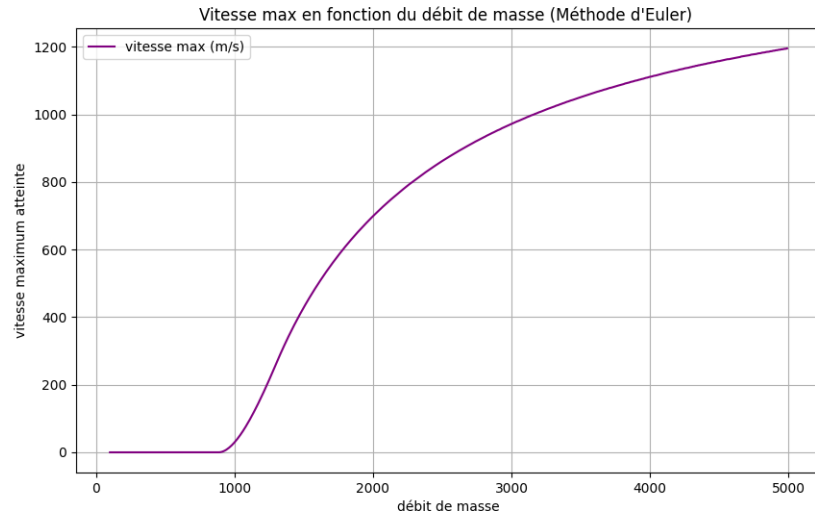
A = aire (du véhicule)

2 Point de situation

À présent, la première étape de la simulation est finie:



— Nous avons obtenu un graphique de la position selon l'axe y en prenant en compte les 4 forces aérodynamiques (les frottements, la poussée, la portance, et la pesanteur). Nous pouvons observer, qu'au bout de 20'000 mètres, la fusée Ariane 62 s'échoue. Nous avons également étudié l'effet qu'avait le débit de masse en observant la vitesse maximale atteinte en faisant varier le débit de masse (valeurs allant de 100 à 5000):



— Afin d’assurer que tout fonctionne correctement, il a fallu mettre quelques tests en place:

3 Tests et problèmes rencontrés

Au niveau du graphique de la hauteur, le code n’est pas trop complexe et ne nécessite pas de tests très intenses, mais afin de vérifier que tout soit normal au niveau des calculs, il se trouve, à la fin de ces-derniers, un:

```
print(v_y)
```

En affichant les vitesses y au fur et à mesure, on peut vérifier qu’elle prend en ampleur et que les valeurs ne sont pas, par exemple, négatives ou oscillantes. Si tout va bien, la vitesse devrait croître et la dernière doit être le maximum, avant que la fusée n’aie plus de carburant. Cette ligne de code permet également de vérifier que les calculs soient faits avant d’afficher le graphique, au cas où il ne s’afficherait pas.

Pour l’étude de la vitesse maximale, le même type de test était utilisé:

```
print(v_max)
```

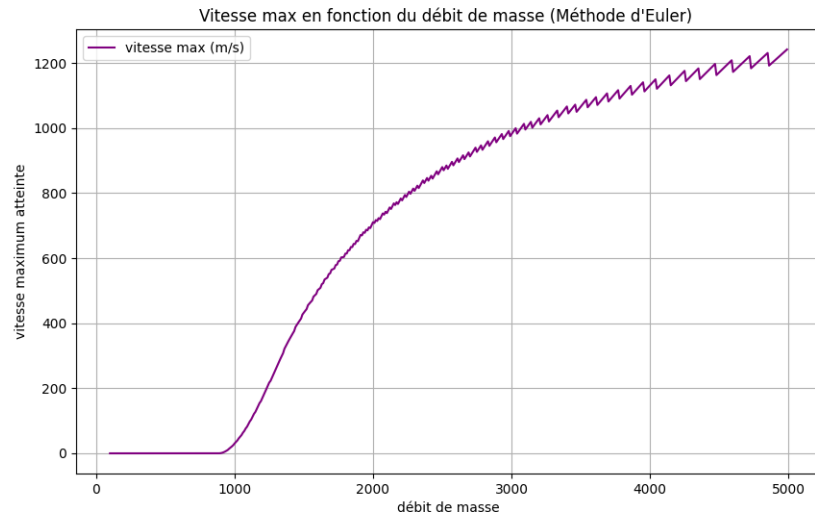
Quelques problèmes s’étaient présentés: D’abord, lors de la vérification, les premières vitesses maximales de la liste affichées étaient négatives. Il s’est révélé que c’était parce qu’il n’y avait pas de “sol” dans la simulation. Alors, si la fusée avait un débit de masse insuffisant pour créer assez de poussée pour décoller, elle “tombait” à cause de la pesanteur. Alors, un “sol” a été introduit dans le code:

```
# gestion du sol (pour éviter les vitesses négatives) si la poussée < la pesanteur
if y <= 0:
```

```
# si on est au sol (ou en dessous)
if v_y < 0:
    # et si la vitesse va vers le bas
    v_y = 0 #le sol nous bloque
    y = 0 # on n'a pas décollé
```

Grâce à cela, les premières vitesses maximales sont nulles jusqu'à avoir assez de poussée pour commencer à croître.

Dans un deuxième temps, le graphique prenait une allure assez étrange:



Cet effet “dents de scie” était créé par le pas de temps énorme (1s) choisi. La méthode d'Euler suppose que l'accélération est constante pendant ces 2 secondes, ce qui est une approximation. De plus, à chaque répétition du code, les erreurs s'accumulent, ce qui peut expliquer beaucoup d'imprécision vers la fin du graphique. Pour améliorer la précision, le pas de temps a été changé pour 0.02s, qui était la plus petite valeur pour laquelle le code tournait.

4 Etape 2

Pour réaliser cette étape, nous prendrons en compte le fait qu'une fusée a plusieurs étages. En effet, à un seul étage, il est impossible de lutter contre l'attraction terrestre. Il n'existe pas de moteur assez puissant, alors en larguant un à un ses étages, la fusée s'allège. Cela lui permet d'atteindre la vitesse minimale pour quitter l'attraction terrestre: 40'000 km/h. Il faut également prendre en compte le changement dans la force gravitationnelle, puisqu'elle dépend de la distance à un corps (ici, la Terre), l'éloignement de la fusée aura un effet dessus.

4.1 Théorie

L'abandon d'étages reprend la théorie de l'étape 1 et sera implémenté dans le code. Voici la loi universelle de la gravitation:

$$F = G \frac{m_1 m_2}{r^2}$$

Légende:

F = force

G = constante gravitationnelle

m_1 = masse de l'objet 1

m_2 = masse de l'objet 2

r = distance entre le centre des deux masses